

One Shot vs. Iterative: Rethinking Pruning Strategies for Model Compression

Mikołaj Janusz^{a,*}, Tomasz Wojnar^a, Yawei Li^b, Luca Benini^b and Kamil Adamczewski^c

^aJagiellonian University

^bETH Zürich

^cWrocław University of Science and Technology

Abstract. Pruning is a core technique for compressing neural networks to improve computational efficiency. This process is typically approached in two ways: one-shot pruning, which involves a single pass of training and pruning, and iterative pruning, where pruning is performed over multiple cycles for potentially finer network refinement. Although iterative pruning has historically seen broader adoption, this preference is often assumed rather than rigorously tested. Our study presents one of the first systematic and comprehensive comparisons of these methods, providing rigorous definitions, benchmarking both across structured and unstructured settings, and applying different pruning criteria and modalities. We find that each method has specific advantages: one-shot pruning proves more effective at lower pruning ratios, while iterative pruning performs better at higher ratios. Building on these findings, we advocate for patience-based pruning and introduce a hybrid approach that can outperform traditional methods in certain scenarios, providing valuable insights for practitioners selecting a pruning strategy tailored to their goals and constraints. Source code is available at <https://github.com/janumiko/pruning-benchmark>.

1 Introduction

As the complexity and scale of tasks for machine learning and computer vision continue to grow, so does the size of neural networks designed to address these tasks [10, 20, 21, 25, 28, 48]. Larger models typically achieve superior performance. However, deploying these large models can be computationally expensive and resource-intensive, making them impractical for environments with limited computational power, such as mobile devices or embedded systems [15, 17, 37]. Pruning is a critical technique that reduces network size without significantly compromising performance [5, 9, 22, 31, 40, 41, 45, 55]. By removing redundant parts of a network, pruning can help achieve results similar to those of fully trained networks, especially in inference scenarios where efficiency is essential.

Pruning techniques are broadly divided into two categories: **one-shot pruning** and **iterative pruning** (Figure 1). One-shot pruning consists of a single cycle of training, pruning, and retraining, as detailed in Section 3. In contrast, iterative pruning involves multiple cycles of these steps, leading to potentially more refined and efficient

network structures. Despite their prevalence, a systematic comparison of these two approaches under different regimes is limited in the literature. Prior research has largely focused on either the criteria for pruning [35, 55] or the specific pruning methods [6, 41, 53], leaving a gap in understanding the relative performance of different pruning strategies under varying conditions.

In this work, we conduct a thorough comparison of one-shot versus iterative pruning across multiple settings. Such a comparison requires isolating the pruning regime as a variable, which fits into the broader view of pruning as a multi-objective optimization problem trading off accuracy, model size, and computational cost. Our controlled experimental setup holds the pruning criterion and target sparsity fixed to achieve this isolation. Furthermore, to properly evaluate the retraining cost, we propose patience-based fine-tuning as a more natural way to gauge fine-tuning duration. Our analysis focuses primarily on vision tasks, where the pruning literature is most extensive. To broaden our investigation, we also present an exploratory case study on a text generation task to explore how these regimes translate to the NLP domain.

For iterative pruning, we introduce a geometric pruning ratio scheduler in addition to the constant pruning ratio scheduler. Unlike the constant scheduler, which prunes a fixed percentage of weights across the entire network, the geometric scheduler prunes a fixed percentage of the remaining weights at each step, progressively removing fewer weights as pruning progresses. Our experiments reveal that the geometric pruning scheduler generally outperforms the constant scheduler.

Additionally, inspired by the respective strengths of one-shot and iterative pruning, we propose a **hybrid few-shot pruning** regime, combining aspects of both methods. This hybrid approach shows advantages over the individual techniques in certain scenarios. Notably, this study does not seek to establish one method as universally superior; instead, it aims to provide guidelines to help practitioners choose the most suitable pruning approach based on specific requirements and constraints.

To summarize, the contributions of this work include:

- Defining the problem of pruning regime selection, and the first broad comparison of one-shot and iterative pruning methods.
- The introduction of a geometric pruning ratio scheduler that prunes a fixed percentage of remaining weights in each pruning step, demonstrating benefits for iterative pruning.
- Proposing a hybrid approach that combines aspects of one-shot and iterative pruning to provide flexibility in specific cases.

* Corresponding Author. Email: mikolaj1.janusz@student.uj.edu.pl

- Broad empirical results that describe preferred settings for each pruning regime, in particular the impact of pruning regime on pruning criteria.

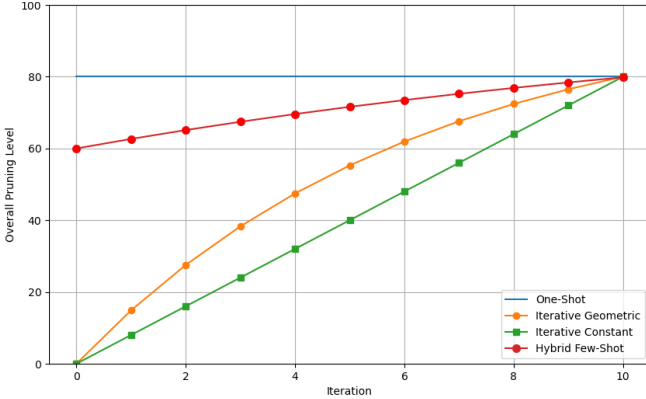


Figure 1: A conceptual illustration of the different pruning regimes. This schematic shows the progression of weight removal for one-shot, iterative constant, and iterative geometric pruning, and is not derived from a specific experiment.

2 Pruning regimes in literature

Pruning is one of the earliest and most established techniques for compressing neural networks. This section attempts to show the extent of the bias in the pruning literature towards proposing new pruning criteria. This bias underscores the importance of the proposed problem, which is broadly orthogonal and shows an alternative direction of pruning research.

Early research demonstrated that training large networks and then removing weights can significantly improve computational efficiency and reduce parameter size [19, 29]. Much of the pruning literature focuses on developing new methods to define pruning criteria, with the most common approaches based on the magnitude of weights [15, 23], sensitivity to derivatives [29, 44], or empirical filter sensitivity [3, 36]. Other methods have been introduced that prune based on neuron similarity [49], activation functions [38], or even more unconventional criteria, such as game-theoretic [2], reinforcement learning [22], or genetic algorithms [18, 40].

Despite this diversity of pruning methods, there is no unified procedure for how networks are pruned. Instead, two main pruning strategies are prevalent in the literature: one-shot pruning and iterative pruning. These approaches differ in whether the network is pruned all at once or in successive stages. However, the impact of this choice on pruning outcomes is not well understood. Each approach also comes with a unique set of parameters that can significantly influence final network performance, such as fine-tuning duration and the number of pruning iterations. Details like the exact percentage of weights removed at each step are often not explicitly provided, leaving gaps in the understanding of how specific pruning configurations affect results. In this work, we argue that these choices are as crucial as pruning criteria and can greatly influence the overall performance of the pruned network. Our study aims to address this gap in the literature, conducting a detailed analysis to clarify how different pruning strategies affect network performance.

In the following sections, we discuss studies that employ both one-shot and iterative pruning, and in Table 1, we summarize common pruning methods and their associated training protocols.

One-shot pruning. One-shot pruning involves a single cycle of pruning followed by retraining, making it a computationally efficient approach widely appreciated in the literature. In this method, node ranking is computed only once, resulting in the final network structure after a single pruning step.

One-shot pruning is often chosen for methods where pruning costs are high. For instance, He et al. [23] removes filters near the geometric median, arguing that these filters are effectively represented by the remaining ones. Calculating the geometric median, however, is computationally intensive. Similarly, Dirichlet pruning [1] requires a training phase to compute parameters of the Dirichlet distribution, which serve as importance weights. CURL [42], on the other hand, removes all unimportant filters across layers in one step using a KL-divergence-based criterion. Li et al. [34] conducted one-shot pruning according to the gradients of the latent vectors in a hypernetwork, which is introduced to guide network pruning.

After the pruning step, retraining is typically performed. However, the optimal length for retraining has not been thoroughly investigated. Instead, many methods fix the retraining duration arbitrarily; for example, CURL retrains the model for 100 epochs.

Iterative pruning. Iterative pruning refines neural network architectures by progressively removing less important parameters or structures while fine-tuning the network to maintain performance. Unlike one-shot pruning, which removes a significant portion of weights or neurons at once, iterative pruning involves multiple cycles of pruning and retraining.

Iterative pruning requires computing parameter importance at every step. Therefore, for large networks, iterative pruning can be computationally prohibitive in cases of unstructured pruning, due to the vast number of individual parameters. However, in smaller networks, iterative pruning dates back to early methods such as Optimal Brain Damage (OBD) and Optimal Brain Surgeon (OBS) [19], which remove unimportant connections based on the second-order derivatives of the loss function with respect to the weights. In second-derivative methods, [19] requires calculating the inverse Hessian matrix, which involves significant computational effort. Given the computational cost of calculating such pruning criteria, only one [52] or a few batches [36] are often used to evaluate parameter sensitivity.

In iterative pruning, a common approach is to use a constant pruning rate—a fixed percentage of weights pruned in each cycle. However, this rate varies across methods. For example, [45] uses Taylor expansions to approximate filter contributions, removing 10 neurons every 30 mini-batches until the target number of pruned neurons is reached, followed by fine-tuning for 25 epochs. In [47], 20% of the lowest-magnitude weights are pruned globally, after which the network is retrained using learning rate rewinding and the original training time. [35] explores sub-architecture optimization by randomly removing 10% of weights, followed by fine-tuning.

The number of elements pruned in each cycle may be determined by various criteria, often treated as hyperparameters. [16] sets a custom threshold, removing elements whose magnitude is below that threshold. [39] prunes channels at each iteration based on Fisher Information scores, allowing the number of pruned channels to vary. [36] uses a hyperharmonic sequence for pruning ratios, where the i -th pruning ratio follows $1 - \frac{1}{(i+1)^\alpha}$. [38] prunes a specified number of channels layer-wise, fine-tuning after each layer’s pruning. Additionally, [43] uses statistics from subsequent layers to prune each layer iteratively, fine-tuning for one or two epochs after each layer is pruned. Li et al. [32] applied group sparsity to the sparsity-inducing matrix and conducted a proximal gradient descent algorithm to progressively prune the network during the optimization of the pruning

procedure.

Pruning at initialization and during training. While this work primarily focuses on post-training pruning using a train-prune-retrain cycle, we acknowledge other types of pruning. Pruning at initialization [14, 30, 51] avoids pruning a fully trained model, instead identifying a smaller subnetwork to train from scratch for comparable performance to the larger model.

Another category involves pruning during training through regularization, which encourages certain parameters to approach zero [46, 54]. For example, [54] introduces scaling factors that selectively scale the outputs of certain CNN structures, applying sparse regularization to these scaling factors to progressively reduce their influence during training.

Unstructured and structured pruning. Pruning networks can be done for individual weights or for structures within the network. Each of them is important in its own regard and is broadly researched. Hence, this benchmark includes tests where both unstructured and structured pruning are considered. We build our benchmark around the Torch-Pruning [13] for structured pruning, and extend it with unstructured pruning using the built-in pruning utilities in PyTorch. The details for how unstructured and structured pruning are done can be found in the Appendix.

3 Pruning regimes

In the context of neural network optimization, pruning regimes can be split into categories based on the following question: *How should we divide the pruning process?* Should we prune all the redundant weights at once? Or should we divide the pruning between different iterations that are separated by modifying the structure of the network and weight updates? In this section, we present rigorous definitions of pruning criteria.

For the purpose of the comparison of the pruning regimes, assume that W is the total number of weights in the neural network and p is the desired pruning percentage, e.g. $p = 0.8$ means a pruning of 80% of weights.

- **One-shot Pruning:** One-shot pruning is a regime where a substantial portion of the network’s weights are removed in a single pruning step, following the process:
 - Initial Training: The neural network is first fully trained to ensure it learns the patterns in the data.
 - Pruning Step: A certain percentage of the least important weights or neurons (based on metrics like magnitude or importance) are pruned in one go.
 - Fine-tuning: After pruning, the network may undergo additional fine-tuning to regain any performance lost from pruning.

The least significant weights are eliminated based on a predetermined importance criterion. The number of weights removed after one-shot pruning is given by:

$$p \times W.$$

- **Iterative Pruning:** Iterative pruning is a process where weights are pruned over multiple iterations, allowing the network to gradually retrain and recover some performance loss. In each iteration the ranking, pruning, and fine-tuning occurs. We define two common approaches to iterative pruning and name them differently to avoid confusion.

- **Iterative Constant:** A constant number of parameters is pruned at each step. Let $steps$ be the number of iterations, then

$$\frac{p \times W}{steps}$$

weights are pruned at each stage. The pruned percentage per step is fixed in relation to the initial number of weights.

- **Iterative Geometric:** A fixed percentage p of the remaining weights is pruned at each step, meaning that as the pruning process progresses, fewer weights are pruned at each iteration. The number of weights at step n is given by the following formula:

$$W \times \left(1 - \frac{p}{steps}\right)^n$$

- **Hybrid pruning:** We propose a new pruning regime, hybrid (few-shot) pruning, which is a combination of the idea of one-shot and geometric regimes. The majority of the weights are removed at the first step and the model is retrained for a longer time. Then for the remaining weights, we perform a more fine-grained geometric-like pruning over several iterations. The hybrid pruning approach can be summarized as follows:

- Apply large pruning ratio p_k to the original network and perform a longer fine-tuning phase.
- Perform iterative geometric pruning with smaller pruning ratio, $p_i \ll p_k$ starting from the state with p_k parameters removed. Proceed until the desired final pruning percentage p .

We elaborate on the hybrid regime and empirical estimates for p_k and p_i in Section 5.5.

4 Pruning regime factors

Pruning regimes, whether one-shot or iterative, involve parameters that significantly impact their effectiveness. To properly evaluate these regimes, it is essential to consider these parameters.

4.1 Retraining

Retraining, or fine-tuning, is a critical step in the pruning process, helping the neural network recover performance after a portion of its weights or channels has been removed. Retraining is necessary because the initial pruning can degrade the model’s accuracy by eliminating parameters that the network previously relied upon for making predictions.

Both one-shot and iterative pruning require retraining, though the duration may vary. Generally, the larger the drop in accuracy, the longer the retraining phase should be. The accuracy drop typically correlates with the number and importance of pruned parameters, leading to longer fine-tuning for one-shot pruning and shorter, repeated fine-tuning phases for each step in iterative pruning.

However, the optimal length of retraining has not been rigorously studied, with methods often using arbitrary values. For instance, [47] fine-tunes the model for the full original training time, which may be computationally excessive since the network is not being trained from scratch. Conversely, in [7, 43], fine-tuning is limited to one epoch after each pruning step in iterative pruning.

Method	structure	regime	step
HRank [38]	structured	iterative (custom)	
ThiNet [43]	structured	iterative (custom)	
SSS [26]	structured	iterative (unspecified)	
Revisiting Random Pruning (RRCP) [35]	structured	iterative (constant)	10%
Fisher information [52]	structured	iterative	
Learning rate rewinding [47]	both	iterative (constant)	20%
Optimal brain damage [29]	unstructured	iterative	
Optimal brain surgeon [19]	unstructured	iterative	
Learning weights and connections [16]	unstructured	iterative	
Taylor Expansion [45]	structured	iterative (constant)	2%
Group Fisher Information [32]	structured	iterative (custom)	
Empirical Sensitivity Analysis [36]	structured	iterative (custom)	
CURL (KL-divergence metric) [42]	structured	one-shot	
Dirichlet Pruning [1]	structured	one-shot	
Geometric Median [23]	structured	one-shot	

Table 1: A list of selected pruning methods and their corresponding training regimes

Patience (early stopping). In this work, we advocate for using patience or early stopping to determine the number of fine-tuning epochs in both one-shot and iterative pruning. Patience allows a dynamic approach to fine-tuning, where the model is trained until a specified criterion (e.g., validation accuracy) no longer improves over a set number of epochs. The best-performing checkpoint is retained. Patience is beneficial because it adapts the fine-tuning duration based on actual performance, whereas fixed epochs may either be insufficient or wasteful. In this study, we use patience as an alternative to a fixed epoch count, defining it as the number of epochs to continue fine-tuning before stopping if no improvement occurs. The details of our patience-based algorithm are outlined in Appendix Algorithm 1. In Appendix Section A we present a comparative study to emphasize the importance of proper selection of both patience-based retraining length and step size.

4.2 Iteration Pruning Rate

In iterative pruning, a key parameter is the iteration pruning rate, which determines the percentage of parameters removed at each step. Assuming a fixed target pruning rate, the iteration pruning rate is defined either by the number of steps or, conversely, the number of steps is determined based on the selected pruning rate.

As discussed in Section 1, the pruning rate varies widely across methods, from as low as 1% to as high as 20–30%. Reflecting on these values, we examine both single-digit and double-digit iterative pruning rates for constant and geometric pruning. In constant pruning, the iterative rate should evenly divide the final pruning rate. In geometric pruning, however, the amount pruned at each step decreases, and the rate is set so that the geometric sum of pruning rates across steps achieves a predefined final pruning level. Further details can be found in the Appendix.

5 Pruning regime empirical evaluation

This section consists of five parts. We first present some main takeaways and then present the detailed experiments. In the second part, we present broad results in Computer Vision and Natural Language Processing settings for the most common magnitude pruning. In the third part, we extend the results to other pruning criteria and highlight that the choice of pruning regime impacts the pruning outcomes

differently when different pruning criteria are applied, a problem broadly overlooked in the literature. In the fourth section, we consider comparing the regimes when the pruning computational budget is fixed. In the final part, based on our analysis of one-shot and iterative regimes, we propose a novel hybrid approach that combines both existing pruning regimes, retaining its strength and producing a more informed and better-performing pruning regime.

Experimental set-up. We perform experiments on several datasets and model architectures. The datasets include vision datasets, CIFAR-10 [27], CIFAR-100, and Imagenet1K [8] and the language dataset TinyStories [11]. The experiments are performed both on convolutional neural networks and transformers, in particular ResNet [21], EfficientNet [50], Visual Transformer [10] and TinyStories-33M [11]. The open-sourced codebase allows for other custom choices. As recommended in [16], we use $1/10^{\text{th}}$ of the original learning rate for the fine-tuning phase.

5.1 Key observations

- One-shot pruning can perform better than iterative pruning for CNNs and lower pruning rates, and iterative pruning is better for transformers and higher rates.
- One-shot pruning typically reduces retraining time compared to iterative pruning by avoiding repeated cycles of pruning and retraining, which is especially helpful when computational resources are limited.
- Iterative geometric pruning is superior to constant iterative pruning in most cases.
- Early stopping ensures optimal fine-tuning time.
- Number of retraining iterations matters significantly.
- Iterative pruning is preferable for second-derivative methods.

5.2 Comparison of one-shot and iterative regimes.

One-shot pruning with patience-based retraining. We demonstrate that one-shot pruning, when paired with an adaptive retraining duration, can be highly effective, surpassing both forms of iterative pruning, as shown in Fig. 2. Our approach to one-shot pruning uses patience-based retraining, allowing the model to stop fine-tuning

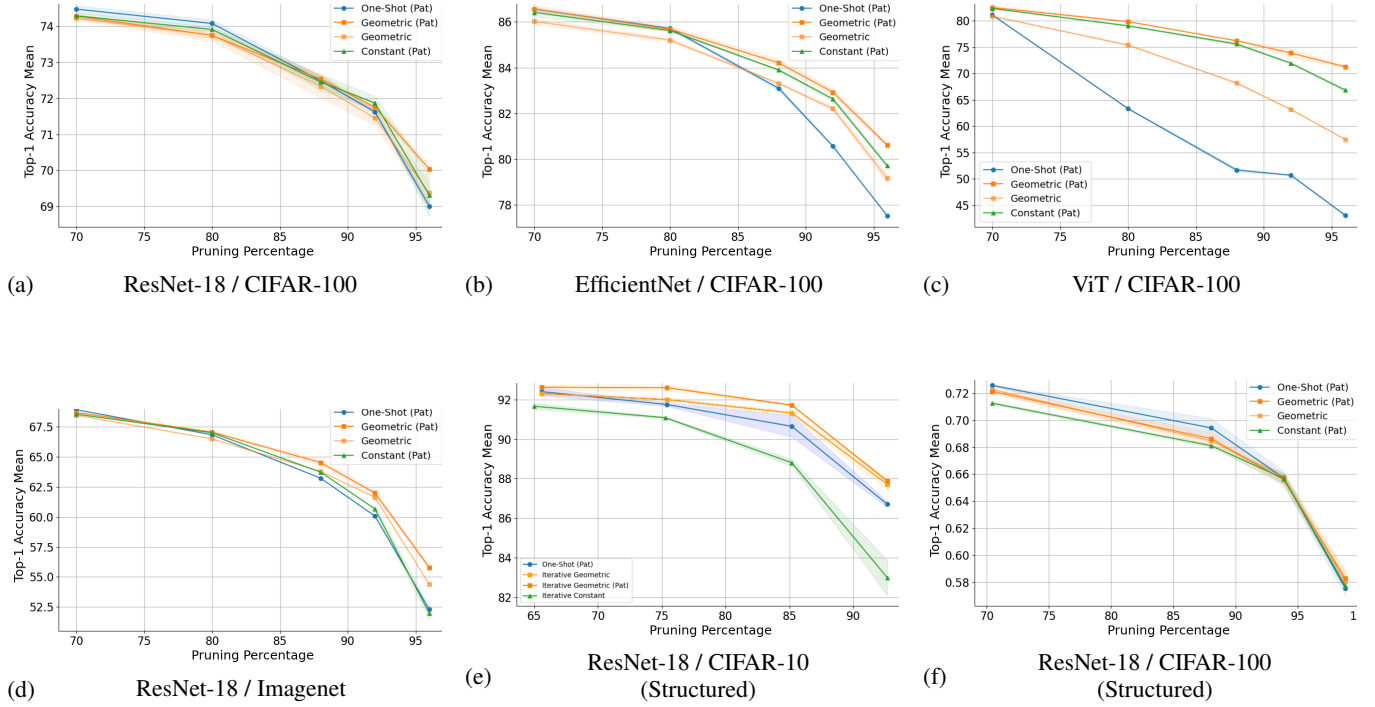


Figure 2: Comparison of pruning regimes across architectures and datasets. Method with (Pat) in the name indicate the patience-based fine-tuning. The performance of one-shot, iterative constant and iterative geometric regimes are plotted. 'Geometric' outperforms 'Constant' in most high-sparsity scenarios. The y-axis represents Top-1 Accuracy (%). See Appendix for fixed-length regimes.

once there is no improvement over a specified number of epochs. This method is more adaptive than using a fixed number of epochs, which may result in either insufficient or excessive retraining. Notably, one-shot pruning consistently outperforms iterative pruning, particularly at pruning rates below 80%.

Iterative pruning with fixed retraining. In contrast, iterative pruning in the literature is often paired with a fixed fine-tuning phase, sometimes limited to as little as one epoch [7]. In our experiments, we test a range of fixed retraining durations for both iterative geometric and iterative constant pruning, selecting the best-performing configuration, which is plotted in Figure 2. Results indicate that short, fixed retraining phases in iterative pruning lead to suboptimal performance. Geometric iterative pruning performs better at higher pruning ratios and in transformer models and structured pruning contexts.

Iterative pruning with patience-based retraining. To enable a fair comparison with one-shot pruning, we propose implementing patience-based fine-tuning for iterative pruning, allowing both methods to benefit from early stopping. Since iterative pruning removes smaller fractions of weights in each step, we set a shorter patience period than in one-shot pruning to maintain efficiency. An ablation study on patience values is included in Section A. As shown in Figure 2, patience-based iterative pruning improves fine-tuning effectiveness over standard iterative pruning, achieving higher accuracies, especially at high pruning ratios where heavily pruned networks are more sensitive to overtraining or undertraining. Iterative pruning is also preferable for transformers.

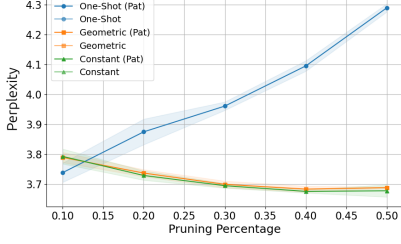
5.2.1 Natural language processing

In addition to computer vision tasks, we also conduct experiments on a natural language processing (NLP) task, specifically text generation. For these experiments, we prune the pre-trained TinyStories-33M language model [12], which is based on GPT-Neo [4]. We use the perplexity metric to evaluate pruning and fine-tuning on the TinyStories dataset. Perplexity measures how well a probabilistic model predicts a sequence of words, with lower perplexity indicating better performance. As in our previous experiments, we explore various pruning schedules and apply patience-based fine-tuning. The results are shown in Fig. 3. Generally, we observe a similar relative performance pattern between pruning regimes: one-shot pruning performs better at lower pruning ratios, while iterative pruning excels at higher compression rates, with iterative pruning showing a notably larger advantage in this context. However, unlike vision tasks, in case NLP models are more sensitive to one-shot pruning, showing performance degradation even when only 10–20% of the parameters are removed. On the other hand, interestingly, we find that in the case of iterative pruning, perplexity decreases as pruning progresses, suggesting that the LLM contains a substantial number of redundant parameters and benefits from pruning.

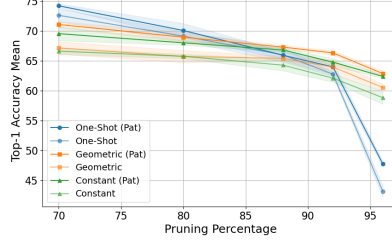
5.3 Pruning methods comparison

We then examine how the choice of pruning criteria influences the selection of a training regime. In this study, we compare three key criteria: magnitude-based, Taylor Expansion, and Hessian-based pruning. The results are presented in Fig. 5.

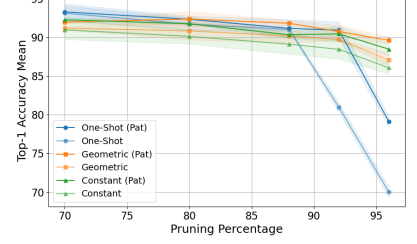
Generally, for lower pruning ratios, the one-shot regime performs better across all criteria except the constant regime. Notably, second-



(a) TinyStories/TinyStories-33M (GPT-Neo)

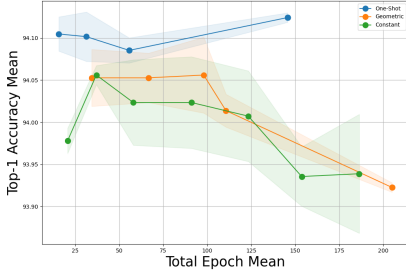


(b) ResNet-18 / CIFAR-100 / Hessian

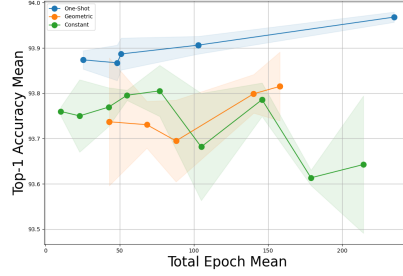


(c) ResNet-18 / CIFAR-10 / Hessian

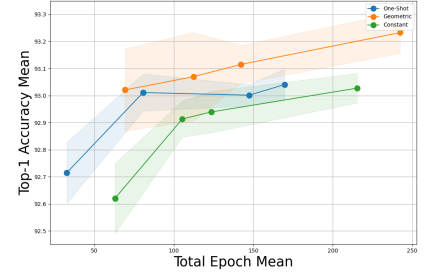
Figure 3: The performance of training regimes for (a) natural language processing TinyStories text generation dataset. Lower perplexity means better performance. (b-c) second-derivative pruning criteria on vision datasets.



(a) Pruning rate 70%



(b) Pruning rate 80%



(c) Pruning rate 92%

Figure 4: The performance of training regimes for fixed computational budget, given in terms of total number of epochs. One-shot is more efficient for pruning rates below 80% while iterative geometric for higher pruning rates.

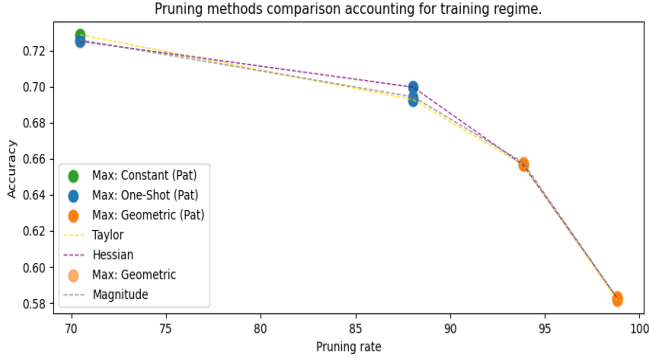


Figure 5: Each dashed line tracks the performance of a single pruning criterion. The coloured dot on the line indicates which regime (one-shot, iterative, etc.) achieved that best result at a given pruning ratio.

order approaches outperform Taylor Expansion at 70% pruning and Hessian-based pruning at 88%. However, at higher pruning rates (over 90%), an interesting conclusion emerges: the pruning regime becomes less significant, as all criteria yield similar performance. In these cases, the iterative geometric approach is preferred across the board.

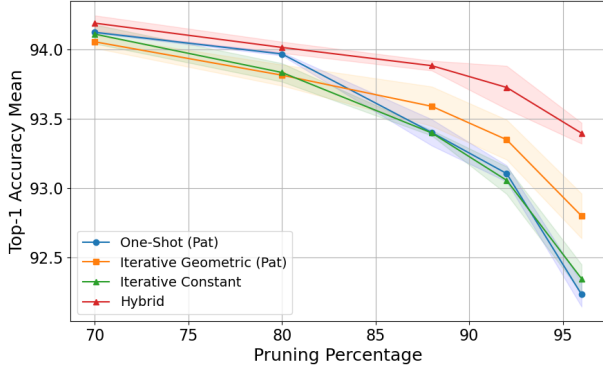
From a computational perspective, these findings are encourag-

ing. The cost of computing pruning rankings varies: it is lowest for magnitude-based pruning and highest for second-order approaches. Since second-order pruning performs better at lower pruning ratios, it is computationally efficient in one-shot scenarios, as the rankings need to be computed only once. Conversely, for higher pruning ratios where iterative pruning is preferable, the choice of criterion becomes less critical. In such cases, magnitude-based pruning is advantageous due to its faster ranking computation.

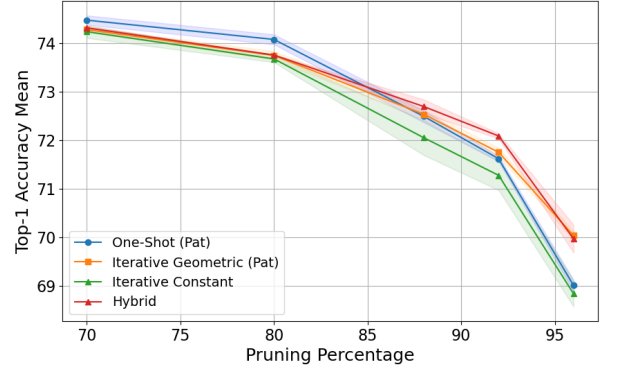
5.4 Retraining Budget

In this section, we consider the retraining budget alongside pruning rate and accuracy. We pose the question: *For a given pruning rate and computational budget, which method yields the best performance?* In Figure 4 we present three plots representing different pruning rates, comparing the budgets used by one-shot and iterative pruning to achieve a given accuracy. The results shown here are based on the ResNet architecture trained on CIFAR-10; additional examples can be found in the Appendix.

The retraining budget is measured in terms of the total number of retraining epochs. For one-shot pruning, this budget corresponds to a single sequence of epochs. For iterative pruning, it represents the sum of epochs over all iterations. As illustrated in Figure 4, one-shot pruning proves to be the most efficient approach for pruning rates up to 80% across all computational budgets, achieving higher accuracy across the range of total epochs. However, at higher pruning rates, iterative pruning shows improved performance, making it the preferred method in these cases.



(a) ResNet-18 / CIFAR-10



(b) ResNet-18 / CIFAR-100

Figure 6: Hybrid approach in comparison with one-shot and iterative pruning.

5.5 Hybrid Regime

The findings from this work indicate that for lower pruning ratios, one-shot pruning is generally more effective than iterative pruning. Building on this insight, we propose a hybrid few-shot approach that combines elements of one-shot and iterative pruning. This hybrid method prunes a large portion of the network in a one-shot-like step, followed by a more refined, geometric pruning strategy. The results, shown in Figure 6, demonstrate that the hybrid approach performs best across nearly all pruning rates, particularly enhancing performance at lower pruning rates. Hybrid pruning leverages the strengths of both one-shot and iterative approaches: it removes the majority of weights in the initial iteration, reducing redundant cycles early in the pruning process, while retaining the precision of geometric iterations at higher pruning rates, where remaining weights carry greater importance and require finer adjustment.

Benchmarking the hybrid approach provides valuable insights into optimal parameter settings. As a general guideline, in the initial step, 60–80% of the target pruning rate p can be pruned (denoted as p_k), followed by retraining with extended patience (approximately 200 epochs). The remaining weights are then pruned iteratively with a rate $p_i \ll p_k$, using diminishing amounts defined by a geometric sequence. For final pruning rates $p < 80\%$, the iterative pruning rate p_k can be around 10%, while for higher pruning rates p_k decreases to about 2%. Fine-tuning then continues with patience set to approximately $\frac{1}{20}$ of the patience used in one-shot pruning.

6 Conclusion

In summary, this study provides a broad evaluation of one-shot and iterative pruning strategies, addressing a critical gap in neural network optimization research. While one-shot pruning is effective at lower pruning ratios, iterative pruning proves superior for higher pruning rates, and arguably transformer architectures and second-derivative pruning criteria. Additionally, our proposed hybrid pruning integrates the strengths of both one-shot and iterative approaches.

This study offers an empirical basis for practitioners to select a pruning regime, including key hyperparameters such as pruning length, incorporating a proposed patience-based approach and step size. Choosing an optimal pruning strategy should be tailored to the specific performance objectives and computational constraints. Future research should further investigate the impact of pruning strategies under different pruning criteria, addressing limitations identi-

fied in this work and refining techniques for more effective pruning regimes.

7 Acknowledgments

We gratefully acknowledge Polish high-performance computing infrastructure PLGrid (HPC Center: ACK Cyfronet AGH) for providing computer facilities and support within computational grant no. PLG/2024/017173. The work of Tomasz Wojnar was supported by the National Centre of Science (Poland) Grant No. 2023/50/E/ST6/00068. The work of Mikołaj Janusz was funded by the "Interpretable and Interactive Multimodal Retrieval in Drug Discovery" project. The „Interpretable and Interactive Multimodal Retrieval in Drug Discovery” project (FENG.02.02-IP.05-0040/23) is carried out within the First Team programme of the Foundation for Polish Science co-financed by the European Union under the European Funds for Smart Economy 2021-2027 (FENG).

References

- [1] K. Adamczewski and M. Park. Dirichlet pruning for neural network compression. *The 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2021.
- [2] K. Adamczewski, C. Sakaridis, V. Patil, and L. Van Gool. Neuron ranking – an informed way to condense convolutional neural networks architecture. In *NeurIPS EMC2 workshop*, 2019.
- [3] C. Baykal, L. Liebenwein, I. Gilitschenski, D. Feldman, and D. Rus. Sipping neural networks: Sensitivity-informed provable pruning of neural networks. *arXiv preprint arXiv:1910.05422*, 2019.
- [4] S. Black, L. Gao, P. Wang, C. Leahy, and S. Biderman. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, Mar. 2021. URL <https://doi.org/10.5281/zenodo.5297715>. If you use this software, please cite it using these metadata.
- [5] C. Chen, F. Tung, N. Vedula, and G. Mori. Constraint-aware deep neural network compression. In *Proceeding of the European Conference on Computer Vision*, pages 400–415, 2018.
- [6] H. Cheng, M. Zhang, and J. Q. Shi. A survey on deep neural network pruning-taxonomy, comparison, analysis, and recommendations. *arXiv preprint arXiv:2308.06767*, 2023.
- [7] E. J. Crowley, J. Turner, A. Storkey, and M. O’Boyle. A closer look at structured pruning for neural network compression. *arXiv preprint arXiv:1810.04622*, 2018.
- [8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255. IEEE, 2009.
- [9] X. Ding, G. Ding, Y. Guo, and J. Han. Centripetal SGD for pruning very deep convolutional networks with complicated structure. In *Proceed-*

- ings of the *IEEE Conference on Computer Vision and Pattern Recognition*, pages 4943–4953, 2019.
- [10] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
 - [11] R. Eldan and Y. Li. Tinstories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.
 - [12] R. Eldan and Y. Li. Tinstories: How small can language models be and still speak coherent english?, 2023. URL <https://arxiv.org/abs/2305.07759>.
 - [13] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang. Depgraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16091–16101, 2023.
 - [14] J. Frankle and M. Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018.
 - [15] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *Proceedings of International Conference on Learning Representations*, 2015.
 - [16] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural network. In *Advances in Neural Information Processing Systems*, pages 1135–1143, 2015.
 - [17] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally. Eie: Efficient inference engine on compressed deep neural network. *ACM SIGARCH Computer Architecture News*, 44(3):243–254, 2016.
 - [18] P. J. Hancock. Pruning neural nets by genetic algorithm. In I. ALEKSANDER and J. TAYLOR, editors, *Artificial Neural Networks*, pages 991–994. North-Holland, Amsterdam, 1992. ISBN 978-0-444-89488-5. doi: <https://doi.org/10.1016/B978-0-444-89488-5.50036-1>. URL <https://www.sciencedirect.com/science/article/pii/B9780444894885500361>.
 - [19] B. Hassibi and D. G. Stork. Second order derivatives for network pruning: Optimal brain surgeon. In *Advances in Neural Information Processing Systems*, pages 164–171, 1993.
 - [20] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1026–1034, 2015.
 - [21] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
 - [22] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han. AMC: AutoML for model compression and acceleration on mobile devices. In *Proceeding of the European Conference on Computer Vision*, pages 784–800, 2018.
 - [23] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang. Filter pruning via geometric median for deep convolutional neural networks acceleration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4340–4349, 2019.
 - [24] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *The Journal of Machine Learning Research*, 22(1): 10882–11005, 2021.
 - [25] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2261–2269, 2017.
 - [26] Z. Huang and N. Wang. Data-driven sparse structure selection for deep neural networks. In *Proceeding of the European Conference on Computer Vision*, pages 304–320, 2018.
 - [27] A. Krizhevsky, V. Nair, and G. Hinton. Cifar-10 (canadian institute for advanced research). URL <http://www.cs.toronto.edu/~kriz/cifar.html>.
 - [28] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, pages 1097–1105, 2012.
 - [29] Y. LeCun, J. S. Denker, and S. A. Solla. Optimal brain damage. In *Advances in neural information processing systems*, pages 598–605, 1990.
 - [30] N. Lee, T. Ajanthan, and P. H. Torr. SNIP: Single-shot network pruning based on connection sensitivity. *arXiv preprint arXiv:1810.02340*, 2018.
 - [31] J. Li, Q. Qi, J. Wang, C. Ge, Y. Li, Z. Yue, and H. Sun. OICSR: Out-in-channel sparsity regularization for compact deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7046–7055, 2019.
 - [32] Y. Li, S. Gu, C. Mayer, L. Van Gool, and R. Timofte. Group sparsity: The hinge between filter pruning and decomposition for network compression. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2020.
 - [33] Y. Li, S. Gu, K. Zhang, L. Van Gool, and R. Timofte. DHP: Differentiable meta pruning via hypernetworks. In *Proceeding of the European Conference on Computer Vision*, pages 608–624. Springer, 2020.
 - [34] Y. Li, W. Li, M. Danelljan, K. Zhang, S. Gu, L. Van Gool, and R. Timofte. The heterogeneity hypothesis: Finding layer-wise differentiated network architectures. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2144–2153, 2021.
 - [35] Y. Li, K. Adamczewski, W. Li, S. Gu, R. Timofte, and L. Van Gool. Revisiting random channel pruning for neural network compression. In *Proceedings of the IEEE International Conference on Computer Vision*, 2022.
 - [36] L. Liebenwein, C. Baykal, H. Lang, D. Feldman, and D. Rus. Provable filter pruning for efficient neural networks. *arXiv preprint arXiv:1911.07412*, 2019.
 - [37] J. Lin, W.-M. Chen, Y. Lin, C. Gan, S. Han, et al. Mucnet: Tiny deep learning on iot devices. *Advances in Neural Information Processing Systems*, 33:11711–11722, 2020.
 - [38] M. Lin, R. Ji, Y. Wang, Y. Zhang, B. Zhang, Y. Tian, and L. Shao. HRank: Filter pruning using high-rank feature map. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1529–1538, 2020.
 - [39] L. Liu, S. Zhang, Z. Kuang, A. Zhou, J.-H. Xue, X. Wang, Y. Chen, W. Yang, Q. Liao, and W. Zhang. Group fisher pruning for practical network compression. In *International Conference on Machine Learning*, pages 7021–7032. PMLR, 2021.
 - [40] Z. Liu, H. Mu, X. Zhang, Z. Guo, X. Yang, T. K.-T. Cheng, and J. Sun. MetaPruning: Meta learning for automatic neural network channel pruning. In *Proceedings of the IEEE International Conference on Computer Vision*, 2019.
 - [41] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell. Rethinking the value of network pruning. In *Proceedings of International Conference on Learning Representations*, 2019.
 - [42] J.-H. Luo and J. Wu. Neural network pruning with residual-connections and limited-data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1458–1467, 2020.
 - [43] J.-H. Luo, J. Wu, and W. Lin. Thinet: A filter level pruning method for deep neural network compression. In *Proceedings of the IEEE international conference on computer vision*, pages 5058–5066, 2017.
 - [44] P. Molchanov, S. Tyree, T. Karras, A. Aila, and J. Kautz. Pruning convolutional neural networks for resource efficient transfer learning. *arXiv preprint arXiv:1611.06440*, 3, 2016.
 - [45] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz. Importance estimation for neural network pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 11264–11272, 2019.
 - [46] C. Oh, K. Adamczewski, and M. Park. Radial and directional posteriors for bayesian deep learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
 - [47] A. Renda, J. Frankle, and M. Carbin. Comparing rewinding and fine-tuning in neural network pruning. *arXiv preprint arXiv:2003.02389*, 2020.
 - [48] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
 - [49] Y. Sui, M. Yin, Y. Xie, H. Phan, S. Aliari Zonouz, and B. Yuan. Chip: Channel independence-based pruning for compact neural networks. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 24604–24616. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/ce6babd060aa46c61a5777902cca78af-Paper.pdf.
 - [50] M. Tan and Q. V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *arXiv preprint arXiv:1905.11946*, 2019.
 - [51] H. Tanaka, D. Kunin, D. L. K. Yamins, and S. Ganguli. Pruning neural networks without any data by iteratively conserving synaptic flow. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/46a4378f835dc8040c8057beb6a2da52-Abstract.html>.
 - [52] L. Theis, I. Korshunova, A. Tejani, and F. Huszar. Faster gaze prediction with dense networks and fisher pruning. *arXiv preprint arXiv:1801.05787*, 2018.
 - [53] H. Wang, C. Qin, Y. Bai, Y. Zhang, and Y. Fu. Recent advances on neural network pruning at initialization. *arXiv preprint arXiv:2103.06460*, 2021.
 - [54] Z. Wang, C. Li, and X. Wang. Convolutional neural network pruning

- with structural redundancy reduction. In *Proceedings of the IEEE/CVF Conference on CVPR*, pages 14913–14922, 2021.
- [55] J. Ye, X. Lu, Z. Lin, and J. Z. Wang. Rethinking the smaller-norm-less-informative assumption in channel pruning of convolution layers. In *Proceedings of International Conference on Learning Representations*, 2018.

APPENDIX

A Ablating pruning retraining length and step size.

As discussed in this work, the retraining duration is a crucial parameter for network pruning, influenced by two main factors: patience and step size. This ablation study highlights the substantial impact of these parameters on pruning performance. Patience-based training allows for better adaptation during retraining cycles. However, as shown in Fig. 7(a-c), choosing an arbitrary patience value can negatively affect performance. Extended retraining may not only waste computational resources but also lead to performance degradation. Similarly, step size is essential in iterative pruning, as illustrated in Fig. 7(d-e). Step size determines the frequency of pruning and fine-tuning cycles, and our findings reveal a non-monotonic trend: both overly small and excessively large steps can reduce performance.

B Pruning criteria further insights

In this work, we experiment with several pruning criteria, including magnitude pruning (Fig. 2), Hessian-based pruning [29] (Fig. 3b-c; see Appendix for structured pruning adaptation), and Taylor expansion-based contribution approximation pruning [45] (see Appendix).

Our primary focus is on magnitude pruning due to its simplicity, effectiveness, reliability, and low computational cost, allowing for extensive benchmarking and experimentation [24]. Magnitude pruning is one of the most widely used pruning criteria across a variety of methods [15, 16, 47].

In this section, however, we want to address the potential similarities and differences in evaluating pruning regimes when alternative criteria are applied. Overall, we find that the relative performance of pruning regimes is largely consistent across different criteria. With the appropriate retraining duration, one-shot pruning performs best up to 80% of the original parameter count, while iterative pruning is preferable at higher compression ratios. Notably, for Hessian-based criteria, one-shot pruning at high pruning rates results in a significant accuracy drop, suggesting iterative pruning may be a more stable solution for second-derivative-based methods.

For second-derivative pruning, the Hessian matrix, which captures the curvature of the loss function, identifies weights in low-curvature regions (small eigenvalues) as good pruning candidates. The experimental results may be explained by the fact that single-step pruning can dramatically alter the loss landscape, rendering the pre-pruning Hessian less accurate in assessing remaining weights. In contrast, iterative pruning enables recalculating the Hessian at each step, ensuring a more precise sensitivity evaluation of the weights retained.

We then expand on the main text, which compares different pruning criteria under various training regimes. We present additional results on structured pruning using two criteria: Hessian-based pruning [29] and Taylor expansion-based contribution approximation pruning [45]. The results, shown in Fig. 8, are largely consistent with the conclusions drawn in the main paper. Specifically, one-shot pruning performs better or comparably up to about a 90% pruning rate, whereas iterative pruning yields better performance at higher compression ratios.

B.1 Further magnitude-pruning results.

Figure 2 presents further comparison of one-shot and iterative pruning across various network architectures and vision datasets. The iterative pruning comes in two types: constant and geometric.

C Early stopping.

In this section, we provide the detailed algorithm for early stopping performed in this paper. We use this algorithm for both one-shot and iterative geometric and iterative constant pruning.

Algorithm 1 Early Stopping Check

Note: This code assumes that a lower metric value indicates better performance (e.g., loss). Otherwise, if a higher metric value is better (e.g., accuracy) the code is run with a reversed comparison.

```
1: procedure EARLYSTOP(metric_value : float)
2:   if metric_value < self.best_metric_value then
3:     self.best_metric_value  $\leftarrow$  metric_value
4:     self.counter  $\leftarrow$  0
5:   else if metric_value > (self.best_metric_value + self.min_delta) then
6:     self.counter  $\leftarrow$  self.counter + 1
7:   end if
8:   if self.counter  $\geq$  self.patience then
9:     return True
10:  end if
11:  return False
12: end procedure
```

D Structured and unstructured pruning

Unstructured pruning. Unstructured pruning involves selectively removing individual weights from the neural network based on criteria such as weight magnitude or their impact on the loss function [15, 14]. This method creates sparse weight matrices with many zero elements, which can significantly reduce the parameter count. However, practical computational gains often require specialized hardware or software optimizations because the remaining weights are irregularly distributed across the network.

Structured pruning. In contrast, structured pruning removes entire components within the neural network, such as filters, channels, neurons, or even layers [22, 55, 41, 32, 33]. This method produces a more compact and regular network structure that retains a dense matrix format, making it easier to optimize on standard hardware. Structured pruning can substantially reduce both model size and computational requirements while maintaining a more organized network. However, achieving high sparsity ratios with structured pruning is more challenging, as it requires removing entire rows or columns rather than individual elements within a weight matrix.

The pruning regimes discussed in the following section apply to both unstructured and structured pruning. However, implementation details may vary due to constraints imposed by the structure of the pruned components.

D.1 Structured pruning pruning ratios

We then provide details on the pruning percentages for each layer in structured pruning. In unstructured pruning, we perform global pruning, allowing pruning to occur freely in any layer. However, applying this approach to structured pruning can lead to pruning collapse, where a layer ends up without any channels. To prevent this, we define a separate pruning ratio for each layer. These ratios are chosen so that the total number of pruned channels across the entire network matches the desired overall pruning percentage. The details of each

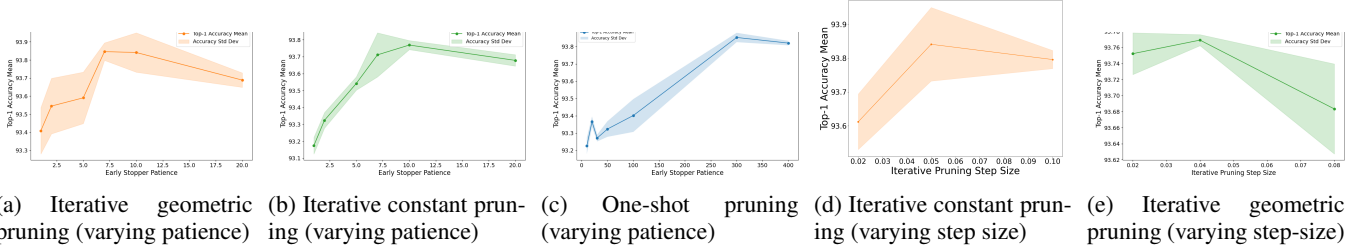


Figure 7: Varying patience and step size (x-axis) impacts the pruning performance (y-axis). In Fig. (a-c) patience is varied and in Fig. (d-e) step size is varied (only for iterative pruning regimes). All experiments are done for pruning rate 88% and CIFAR-10 / ResNet-18.

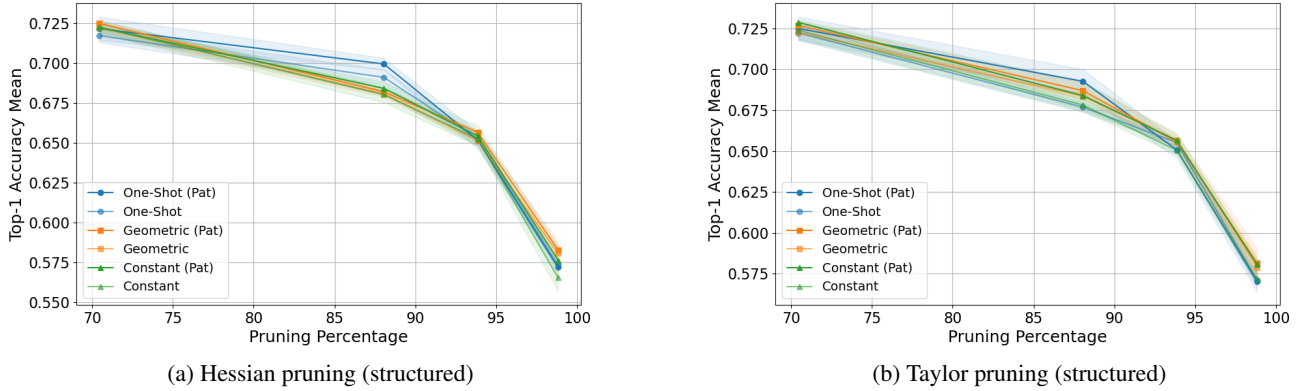


Figure 8: Second-derivative (Hessian) pruning criteria. Iterative vs. one-shot pruning for CIFAR-100 and CIFAR-10.

layer’s pruning ratio and the final pruning percentages are given in Table 2.

E Hybrid regime experimental details

In the hybrid regime experiments, we used the same configuration as in the ResNet-18 experiments on CIFAR-100 and CIFAR-10 datasets. The hybrid regime consisted of an initial one-shot pruning step to a value of $p\%$, followed by iterative geometric steps with a ratio of p_i until reaching the desired total pruning percentage. The first iterative step begins at $p_k\%$. For the hybrid regime, we used different patience values for the one-shot part and the iterative geometric part. We tested all the configurations provided in Tables 3 and 4. For the sake of preciseness we provide the exact pruning percentages p_k which were used in the iterative phase of the hybrid regime; however in this phase, we aimed to test a set of iterative percentages from the set, $p_k = 0.02, 0.05, 0.10$. The adjustments were necessary to obtain the exact final pruning ration p and the fair comparison with other pruning methods.

F Experiments set-up

F.1 Dependencies

The technical setup for the experiments included the following dependencies:

- Python 3.11
- CUDA 12.1
- PyTorch 2.2

- Torchvision 0.17.0
- timm 0.9.16
- torch-pruning 1.4.2

Experiments were conducted mainly on NVIDIA A100 and RTX 2080ti GPU’s.

F.2 Dataset transformations

Cifar-10 and Cifar-100 transformations include normalization (values are located in the code repository), random crop of size 32×32 with padding 4 and random horizontal flip. The images were also resized to higher resolution for some models. ImageNet1K was normalized, resized and cropped.

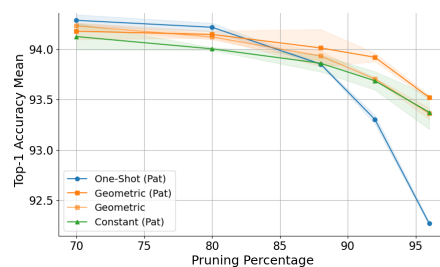
F.3 Checkpoints

The checkpoints for the models used in the experiments are shared here: <https://www.dropbox.com/scl/fo/u0d8a087o3c2ynzpb6chd/AJz5w2ozXzcrBzxUwXVMiYM?rlkey=gag0w2r89kmt1huek6zsy9re2&st=4guxofag&dl=0>.

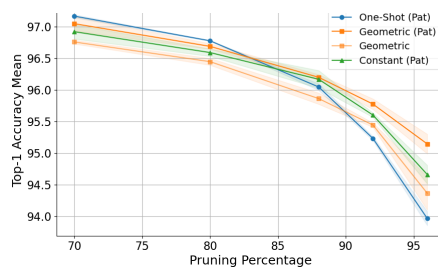
F.4 Parameters

All experiments were conducted using the SGD optimizer with the following parameters:

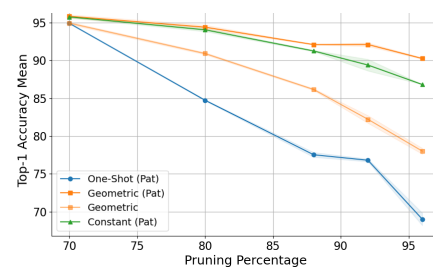
- Learning rate: 0.01
- Momentum: 0.9



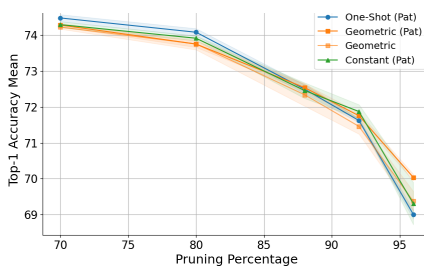
(a) ResNet-18 / CIFAR-10



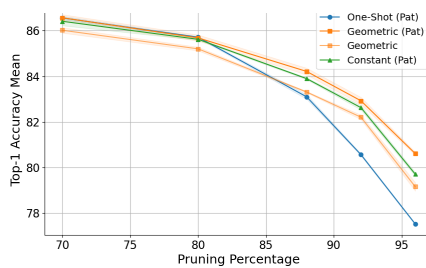
(b) EfficientNet / CIFAR-10



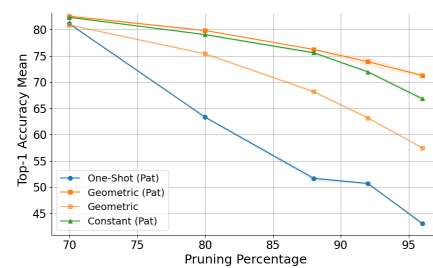
(c) ViT / CIFAR-10



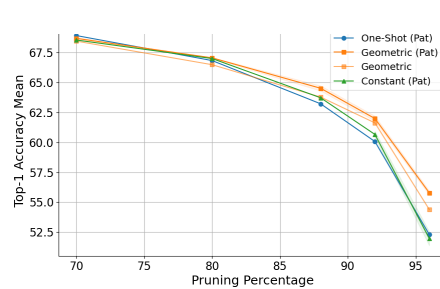
(d) ResNet-18 / CIFAR-100



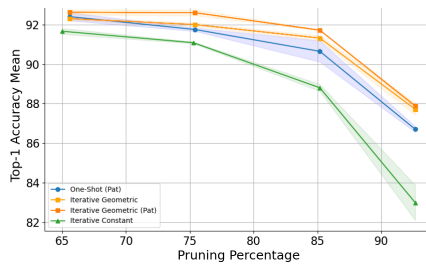
(e) EfficientNet / CIFAR-100



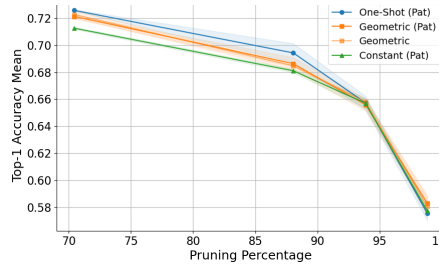
(f) ViT / CIFAR-100



(g) ResNet-18 / Imagenet



(h) ResNet-18 / CIFAR-10 (Structured)



(i) ResNet-18 / CIFAR-100 (Structured)

Figure 9: Comparison of one-shot and iterative pruning across various network architectures and vision datasets.

Layer Name	Conv1 (%)	Layer1 (%)	Layer2 (%)	Layer3 (%)	Layer4 (%)	Pruning Ratio (%)
Model 1	20	20	30	40	50	69.61
Model 2	50	50	60	70	80	93.27
Model 3	40	40	50	60	70	85.25
Model 4	65	65	75	85	95	97.63

Table 2: Pruning percentages for layers and corresponding pruning ratios.

Table 3: Pruning Hybrid Scheduler Parameters

One-shot step	Iterative step	Target pruning value
0.5	0.01842	0.7
0.6	0.01741	0.7
0.5	0.04365	0.7
0.6	0.03451	0.7
0.5	0.07168	0.7
0.6	0.1	0.7
0.5	0.01962	0.8
0.6	0.01842	0.8
0.7	0.01741	0.8
0.5	0.04968	0.8
0.6	0.04365	0.8
0.7	0.03451	0.8
0.5	0.08531	0.8
0.6	0.07168	0.8
0.7	0.05132	0.8
0.5	0.01972	0.88
0.6	0.01914	0.88
0.7	0.01965	0.88
0.8	0.01654	0.88
0.5	0.04668	0.88
0.6	0.04585	0.88
0.7	0.0484	0.88
0.8	0.04083	0.88
0.5	0.09118	0.88
0.6	0.07884	0.88
0.7	0.09446	0.88
0.8	0.08	0.88

- Weight decay: 0.0005

The batch size was set to 512, consistent across all pruning experiments. The training data was shuffled for every run.

F.4.1 CIFAR-100 / ResNet18

We used the ResNet-18 model from the following GitHub repository: <https://github.com/kuangliu/pytorch-cifar/blob/master/models/resnet.py>

Before pruning, the model was trained for 328 epochs with the following configuration:

- SGD optimizer with learning rate: 0.1, momentum: 0.9, and weight decay: 0.0005
- Linear scheduler for 100 epochs with start factor: 0.01
- After 100 epochs: CosineAnnealingWarmRestarts with $T_0 = 50$, $T_{\text{mult}} = 2$, and $\eta_{\text{min}} = 1 \times 10^{-5}$
- Early stopping with patience of 100 epochs

Before pruning, the top-1 accuracy was 74.64%.

F.4.2 CIFAR-10 / ResNet18

The same ResNet-18 model as for CIFAR-100 was used. Before pruning, the model was trained for 226 epochs with the same configuration as the CIFAR-100 experiment. Before pruning, the top-1 accuracy was 94.14%.

Table 4: Pruning Hybrid Scheduler Parameters 0.92 - 0.99

One-shot step	Iterative step	Target pruning value
0.5	0.01997	0.92
0.6	0.0191	0.92
0.7	0.01893	0.92
0.8	0.0181	0.92
0.5	0.04831	0.92
0.6	0.04706	0.92
0.7	0.04848	0.92
0.8	0.04172	0.92
0.5	0.08679	0.92
0.6	0.09191	0.92
0.7	0.07948	0.92
0.8	0.06192	0.92
0.5	0.01968	0.96
0.6	0.01922	0.96
0.7	0.01987	0.96
0.8	0.01919	0.96
0.5	0.04629	0.96
0.6	0.04838	0.96
0.7	0.04895	0.96
0.8	0.04265	0.96
0.5	0.0976	0.96
0.6	0.08539	0.96
0.7	0.0955	0.96
0.8	0.08348	0.96
0.5	0.01961	0.99
0.6	0.01958	0.99
0.7	0.01994	0.99
0.8	0.01897	0.99
0.5	0.04696	0.99
0.6	0.04823	0.99
0.7	0.04775	0.99
0.8	0.04127	0.99
0.5	0.09171	0.99
0.6	0.09413	0.99
0.7	0.08206	0.99
0.8	0.1	0.99

F.4.3 ImageNet / ResNet18

We used the ResNet-18 model from the PyTorch torchvision library with pretrained weights. During fine-tuning in the pruning phase, images were resized to 256×256 and cropped to 224×224 . Before pruning, the top-1 accuracy was 68.91%.

F.4.4 CIFAR-100 / EfficientNet

We used the EfficientNet V2-S model from the PyTorch torchvision library with pretrained weights (IMAGENET1K_V1). Before pruning, the model was trained for 132 epochs with the following configuration:

- SGD optimizer with learning rate: 0.1, momentum: 0.9, and weight decay: 0.0005
- Linear scheduler for 10 epochs with start factor: 0.01
- After 10 epochs: CosineAnnealingWarmRestarts with $T_0 = 10$, $T_{\text{mult}} = 2$, and $\eta_{\text{min}} = 1 \times 10^{-5}$

- Early stopping with patience of 80 epochs
- Images resized to 128×128

Images were resized during fine-tuning in the pruning phase. Before pruning, the top-1 accuracy was 87.53%.

F.4.5 CIFAR-10 / EfficientNet

The same EfficientNet V2-S model as for CIFAR-100 was used. Before pruning, the model was trained for 152 epochs with the same configuration as the CIFAR-100 experiment. Before pruning, the top-1 accuracy was 97.88%.

F.4.6 CIFAR-100 / ViT

We used the ViT small patch16 224 model from the timm library with pretrained weights (vit small patch16 224 augreg in1k). Before pruning, the model was trained for 18 epochs with the following configuration:

- SGD optimizer with learning rate: 0.1, momentum: 0.9, and weight decay: 0.0005
- Linear scheduler for 10 epochs with start factor: 0.01
- After 10 epochs: CosineAnnealingWarmRestarts with $T_0 = 10$, $T_{\text{mult}} = 2$, and $\eta_{\text{min}} = 1 \times 10^{-5}$
- Early stopping with patience of 50 epochs
- Images resized to 224×224

Images were resized during fine-tuning in the pruning phase. Before pruning, the top-1 accuracy was 88.16%.

F.4.7 CIFAR-10 / ViT

The same ViT small patch16 224 model as for CIFAR-100 was used. Before pruning, the model was trained for 19 epochs with the same configuration as the CIFAR-100 experiment. Before pruning, the top-1 accuracy was 98.11%.